

“Pre-training as we know it will end” — Sutskever at NeurIPS 2024

Sutskever, I. (2024). Test-of-Time Award talk, NeurIPS 2024, Vancouver, 10 December 2024 — reflecting on the 2014 paper “Sequence to Sequence Learning with Neural Networks.”

ASSIGNED READING

Watch the talk, or read a transcript (≈ 20 min)

~20 min

» ~20 min • this summary + reasoning notes ~18 min

Note on currency. This is a December 2024 talk — deliberately forward-looking and, in places, speculative. Treat the historical part as settled and the predictions (“pre-training will end”; what superintelligence will be like) as one leading researcher’s informed opinion, not fact. It is here to frame the scaling-and-emergence beat in C1.

KEY TAKEAWAY

Ten years of progress came from a simple recipe — a big neural network trained on a lot of data with a lot of compute (“pre-training”). Sutskever argues that era is ending, not because the idea is wrong, but because we are running out of the one ingredient that doesn’t grow: data. “We have but one internet.” Compute keeps growing; data does not. What comes next, he suggests, leans on agents, synthetic data, and more reasoning at inference time — and, eventually, systems that are genuinely superintelligent: agentic, reasoning, self-aware, and far less predictable than today’s models.

WHY IT MATTERS FOR THIS COURSE

This is the “scaling and emergence” half of C1, told by someone central to the story. It answers the C1 question “what did scale change?”: scaling pre-training is what turned narrow models into general ones, and new abilities appeared that were never explicitly trained (“emergence”). It also sets up why the field is pivoting toward prompting, tool-use and reasoning (C2) and toward economics (C3): if you can’t keep making the model better just by feeding it more data, you extract more from the model you already have — largely through how you prompt it and how much you let it “think.”

Feeds Class 3: C1 (scaling laws • emergence • why scale mattered) • motivates C2 (getting more from a fixed model) • C3 (compute vs data economics).

MAIN POINTS

The 2014 idea that started it

- **The problem in 2014.** Neural networks then required a fixed-size input and produced a fixed-size output — fine for “is this image a cat,” useless for language. “The cat sat on the mat” → “Le chat était assis sur le tapis” is six tokens in, seven out, with no fixed alignment between them. Translation was therefore done by large hand-engineered statistical pipelines — phrase tables, alignment models, reordering rules, a separate language model — each component built and tuned separately.
- **The idea: encoder → vector → decoder.** Sutskever, Vinyals and Le used two recurrent networks — Long Short-Term Memory networks (LSTMs). The **encoder** reads the source sentence one token at a time and compresses the whole of it into a single fixed-length vector. The **decoder** takes that vector and generates the target sentence one token at a time, each token conditioned on the vector and on everything it has already produced. One model, trained end-to-end on sentence pairs by gradient descent. No hand-built components.
- **Why it mattered.** Three things, in ascending order of importance. (i) **It worked** — on English→French it was competitive with, and as an ensemble beat, the mature statistical systems; the first time a purely neural approach did so on a serious translation benchmark. (ii) **It collapsed a whole class of problems into one** — translation, summarisation, question answering, speech transcription, code generation and dialogue all became the same problem: sequence in, sequence out, learned end-to-end. Every chat interface you use is a descendant of that framing. (iii) **It set the bet the next decade ran on** — if a large neural network can learn a hard task end-to-end, then making it bigger and feeding it more data will keep making it better. That is the bet this talk is announcing the end of.
- **The bottleneck — and the fix (→ Reading 1).** Forcing an entire sentence through one fixed-length vector is a choke point, and long sentences degraded badly; the authors resorted to **reversing the word order of the source sentence** as a hack to shorten the path between related words. The real fix was **attention** (Bahdanau, Cho and Bengio, also 2014): let the decoder look back at every encoder state and weight them, instead of trusting one summary vector. By 2017, “Attention Is All You Need” had removed the recurrence entirely and kept only attention — the Transformer Karpathy builds in **Reading 1**. Read the pair this way: **Reading 2 is the bet that scale works; Reading 1 is the architecture that made the bet payable.**

- **Scaling laws.** Bigger models trained on more data with more compute get predictably better. That regularity is why labs kept scaling — the returns were reliable enough to justify the spend.

The claim: pre-training will end

- **Peak data.** Compute grows (better hardware, more chips), but the total useful text on the internet is finite — Sutskever calls data “the fossil fuel of AI.” Once you’ve trained on essentially all of it, that lever stops moving. “We’ve achieved peak data and there’ll be no more.”
- **So the recipe must change.** He points to three directions the field is already taking: agents (systems that act, not just answer), synthetic data (models generating their own training data), and inference-time compute (letting a model reason longer at answer time instead of just being bigger).

Emergence and reasoning

- **New abilities appear with scale.** Capabilities like translation, coding and in-context learning were not hand-built; they emerged as models grew. This is the “emergence” idea C1 introduces.
- **Reasoning brings unpredictability.** He argues that as systems reason more, they become less predictable — genuine reasoning can reach conclusions its designers didn’t foresee (his analogy: a strong chess engine surprising even expert players).

The long horizon

- **Superintelligence, eventually.** His stated end-point: systems that are truly agentic, that reason, that understand from limited data, and that are self-aware — qualitatively different from today’s models. Offered as a direction of travel, with the timeline left open.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to get you started; the fuller reasoning follows after all three, so you can arrive at a view on your own.

- Sutskever says compute keeps growing but data doesn’t. If a model can’t easily get smarter by training on more data, what does that imply for how much your prompt and your context (Weeks 2–3) matter?

Clue. Think about which lever is still free to pull. If the model everyone can rent stops improving quickly, then everyone is building on roughly the same model — so the difference between two products has to come from somewhere else. Ask yourself where that somewhere else could be: what you put in the prompt, what you retrieve into the context window, what you fine-tune on, and how rigorously you measure the result. Notice that three of those four are things a small team can do without a data centre. That is the argument for the whole of Capsules 2 and 3.

- “Emergence” means abilities appear with scale that no one explicitly programmed. Is that reassuring or worrying for someone deploying such a model in your industry — and why?

Clue. Resist picking a side — the honest answer is both, and the useful part is what separates them. Reassuring: you get capabilities nobody had to pay to build. Worrying: if nobody designed the capability, nobody wrote a specification for it, so nobody can tell you precisely when it will fail. Now make it concrete for your own industry: what would you have to test before you were comfortable? Then notice the trap — you cannot write a test for a behaviour you did not know to look for. That gap is exactly why Capsule 2 makes you write an eval set, and why the Med-PaLM team wrote adversarial questions on purpose.

- He predicts more “reasoning at inference time.” In Week 3 you’ll pay per token. What’s the tension between letting a model “think longer” and what it costs to run?

Clue. Start from what “thinking” physically is here. Reasoning at inference time is written out as tokens — it is not free contemplation, it is billable text, and output tokens are the expensive kind. So “let it think longer” literally means “make it generate more.” Then ask the second-order question: who benefits from it? Longer reasoning helps on hard, multi-step problems and is largely wasted on easy ones — which suggests you should not buy it uniformly for every request. Hold that thought: Capsule 3 puts real prices on it, and the Med-PaLM brief shows a team that could afford the most expensive version of this and still rationed it.

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so this section is written to let you settle the argument for yourself. If your view differs and you can defend it, bring it.

Q1 — If data stops being the lever, what is?

Sutskever’s claim is narrow and worth stating precisely: compute keeps growing, but the stock of useful text does not — “we have but one internet.” If that is right, the returns from simply training on more data flatten, and the frontier models available to you and to your competitor converge. Everything that used to be decided by who had the bigger training run then gets decided somewhere else: the instructions you give (Capsule 2’s ladder), the material you put into the context window (Week 2’s retrieval), the data you own and can fine-tune on, and — least

glamorous, most decisive — whether you measure your system well enough to improve it deliberately. Note the strategic reading too: proprietary data and a rigorous eval set become durable advantages precisely because the model itself stops being one. That is also the argument the AI Wars brief makes from the business side.

Q2 — Emergence: reassuring, worrying, or both?

Both, and the distinction that matters is between capability and guarantee. Emergence is a gift on the capability side — translation, coding and in-context learning arrived without anyone specifying them, which is why a general model can do useful work in your industry without being built for it. It is a problem on the guarantee side, because a capability nobody designed is a capability nobody scoped, documented or bounded. You cannot read a specification to find the edges; you have to discover them empirically. The practical consequence is that assurance shifts from design-time to test-time: instead of proving the system correct, you measure it against cases you wrote. And since your test set only covers failures you imagined, the honest posture is to keep expanding it, deliberately including cases meant to break the thing — which is what the Med-PaLM team did with 240 adversarial questions, and what your ten eval cases do in miniature.

Q3 — Thinking longer, priced.

The tension is direct, because inference-time reasoning is not a different kind of activity — it is more tokens. A model that “thinks step by step” writes those steps, and generated tokens are billed at the higher rate and add latency. So the cost of an answer becomes a choice rather than a constant, and the interesting question stops being “can it reason?” and becomes “on which requests is reasoning worth buying?” The sane answer is: not uniformly. Spend it where problems are genuinely multi-step, where a wrong answer is expensive, and where you can measure that the extra spend actually improves the result. Withhold it on easy, high-volume traffic. Two data points to carry forward: Google reserved its most expensive technique for multiple-choice evaluation and said the reason was resource cost; and in Capsule 3 you will price your own prompt and see how quickly a third of a cent becomes a strategy problem.

SOURCE LINKS

Talk — full video (easiest route) · x.com/vincentweisser/status/1867719020444889118

2014 paper (the award winner) · arxiv.org/abs/1409.3215

Talk transcript · github.com/shun-liang/readable-talks-transcriptions (*text mirror — you have to dig into the repo folders*)

Companion · **Reading 1**, Karpathy “Let’s Build GPT” — Reading 1 is the mechanism (how it works), Reading 2 the trajectory (where it came from, where it is going). Read them as a pair.

CAVEATS

1. This summary is not a substitute for the talk. It is a map, not the territory — watch or read the source; it is the authority.
2. This summary was generated and fact-checked using Anthropic models. The predictive claims are Sutskever’s opinions, not established results — treat them critically.